



RAPPORT FINAL

Application de Gestion de Bibliothèque Universitaire

Matière	Génie Logiciel
Filière	Informatique – Licence 3 – Semestre 5
Enseignant	Dr OUEDRAOGO Moïse
Date	Mai 2026

TABLE DES MATIÈRES

1. Introduction
 2. Analyse des exigences
 3. Modélisation et Conception
 4. Architecture et Design Patterns
 5. Implémentation et Tests
 6. Gestion Agile Scrum
 7. Difficultés rencontrées et leçons apprises
 8. Conclusion
-

1. INTRODUCTION

1.1 Contexte

L'Université Norbert Zongo a initié un projet de modernisation de sa bibliothèque universitaire. L'objectif est de passer d'une gestion manuelle à une solution informatisée complète permettant de gérer efficacement les ressources documentaires, les emprunts, les réservations et les interactions entre les différents acteurs.

1.2 Objectifs du projet

Ce projet vise à concevoir, modéliser, développer et tester une application web professionnelle en appliquant l'ensemble des concepts du Génie Logiciel étudiés durant le cours :

- Analyse et spécification des exigences
- Modélisation UML complète (cas d'utilisation, classes, séquences)
- Architecture en couches avec Spring Boot
- Application des principes SOLID et Design Patterns
- Gestion de projet Agile avec Scrum
- Tests unitaires et d'intégration

1.3 Méthodologie

Le projet a été mené selon la méthodologie Agile Scrum avec 4 sprints. L'équipe de 6 étudiants a travaillé avec un Scrum Master et un Product Owner. Les outils utilisés sont GitHub pour le versionnement et la gestion du backlog.

1.4 Technologies

Backend	Java 17, Spring Boot 4.0.6
Base de données	PostgreSQL 16
ORM	Spring Data JPA / Hibernate
Sécurité	Spring Security + JWT
Frontend	Thymeleaf + Bootstrap 5
Tests	JUnit 5, Mockito, MockMvc
Déploiement	Render (Docker)
Versionnement	Git + GitHub

2. ANALYSE DES EXIGENCES

2.1 Acteurs du système

Le système comporte quatre types d'acteurs :

Visiteur : Utilisateur non authentifié pouvant consulter le catalogue et effectuer des recherches.

Étudiant : Utilisateur authentifié pouvant emprunter (max 3 ouvrages), réserver, consulter son historique et recevoir des notifications.

Bibliothécaire : Gestionnaire du catalogue, des emprunts, des retours et des pénalités.

Administrateur : Super-utilisateur gérant les comptes, les statistiques et la configuration du système.

2.2 Exigences fonctionnelles

31 exigences fonctionnelles ont été identifiées et réparties en 8 catégories :

- Gestion du catalogue (4 exigences)
- Recherche d'ouvrages (4 exigences)
- Gestion des emprunts (5 exigences)
- Gestion des réservations (5 exigences)
- Gestion des pénalités (4 exigences)
- Notifications (3 exigences)
- Gestion des utilisateurs (3 exigences)
- Rapports et statistiques (3 exigences)

2.3 Users Stories

20 Users Stories ont été rédigées selon le format Agile standard "En tant que... je veux... afin de..." et priorisées :

- 13 de priorité Haute
- 7 de priorité Moyenne

Exemples de Users Stories clés :

- US04 : En tant qu'Étudiant, je veux emprunter un livre disponible (max 3, 14 jours) afin d'accéder aux ressources documentaires
- US08 : En tant qu'Étudiant, je veux réserver un ouvrage indisponible (file FIFO) afin de l'obtenir dès son retour
- US11 : En tant que Bibliothécaire, je veux valider un retour et déclencher le calcul automatique des pénalités

2.4 Critères d'acceptation

Chaque User Story a été accompagnée de critères d'acceptation précis. Par exemple pour US04 :

- CA1 : Blocage si quota de 3 emprunts atteint
- CA2 : Blocage si compte suspendu avec affichage de la date de fin
- CA3 : Calcul automatique de la date de retour à J+14
- CA4 : Mise à jour du stock et changement de statut si stock = 0

2.5 Exigences non fonctionnelles

19 exigences non fonctionnelles couvrant :

- Performance (temps de réponse < 2s)
- Sécurité (authentification JWT, hashage BCrypt)
- Ergonomie (interface responsive)
- Disponibilité ($\geq 99\%$)
- Qualité du code (SOLID, ≥ 4 Design Patterns, tests $\geq 65\%$)

3. MODÉLISATION ET CONCEPTION

3.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation modélise les interactions entre les 4 acteurs et le système. Il comporte 16 cas d'utilisation principaux avec des relations d'inclusion (<>) et d'extension (<>).

Relations clés :

- UC07 (Valider retour) inclut UC11 (Calcul pénalités)
- UC11 inclut UC12 (Bloquer compte)
- UC08 (Réserver) étend UC04 (Emprunter) quand stock = 0
- UC12 étend UC04 et UC08 pour la vérification de suspension

3.2 Diagramme de Classes

Le diagramme de classes modélise la structure statique du système avec 3 packages principaux :

Package Utilisateurs : Classe abstraite Utilisateur avec héritage vers Etudiant, Bibliothécaire et Administrateur.

Package Documents : Séparation claire entre Ouvrage (concept) et Exemplaire (objet physique). Un Ouvrage possède plusieurs Exemplaires.

Package Opérations : Emprunt, Reservation et Penalite matérialisent les transactions métier avec leurs statuts respectifs.

La séparation Ouvrage/Exemplaire est une décision de conception importante qui permet de gérer finement les stocks physiques.

3.3 Diagrammes de Séquence

Trois scénarios critiques ont été modélisés :

Scénario 1 – Emprunt nominal : Flux complet de la demande d'emprunt avec vérifications (quota, suspension, disponibilité) et création en statut "DEMANDÉ" en attente de validation par le bibliothécaire.

Scénario 2 – Retour avec retard : Enchaînement automatique UC07 → UC11 → UC12 avec calcul des jours de retard, création de la pénalité, suspension du compte, notification email et traitement de la file d'attente.

Scénario 3 – Réservation FIFO : Vérifications multiples (suspension, possession, doublon), création avec horodatage, calcul de la position dans la file.

4. ARCHITECTURE ET DESIGN PATTERNS

4.1 Architecture en couches

L'application suit une architecture en 4 couches :

Couche Présentation : Controllers REST, DTOs, Mappers. Point d'entrée des requêtes HTTP.

Couche Métier : Services et interfaces. Logique métier et règles de gestion.

Couche Domaine : Entités JPA, Enums, Exceptions. Modèle persistant.

Couche Infrastructure : Repositories, Security, Configuration.

Une couche transversale gère les exceptions globales et le logging.

4.2 Principes SOLID

S – Responsabilité Unique	Chaque service a une responsabilité unique (EmpruntService, PenaliteService, NotificationService).
O – Ouvert/Fermé	Les interfaces de service permettent l'extension sans modification du code existant.
L – Substitution de Liskov	Les sous-classes (Etudiant, Bibliothecaire, Administrateur) peuvent remplacer Utilisateur.
I – Ségrégation des Interfaces	Interfaces fines et spécifiques (IEmpruntService, IReservationService).
D – Inversion des Dépendances	Injection par constructeur, les contrôleurs dépendent des interfaces, pas des implémentations.

4.3 Design Patterns

6 Design Patterns ont été implémentés (minimum exigé : 4) :

#	Pattern	Framework	Rôle
1	Repository	Spring Data JPA	Abstraction de la persistance
2	DTO	–	Séparation entre entités JPA et objets API
3	Singleton	Spring IoC	Beans Spring gérés par le conteneur IoC
4	Observer	Spring Events	Découplage entre retour et notifications
5	Template Method	–	Méthode peutEmprunter() abstraite dans Utilisateur
6	Factory Method	–	Création encapsulée des pénalités

4.4 Vue de Déploiement

L'application est déployée sur la plateforme Render avec :

- Web Service Spring Boot conteneurisé via Docker
- Base de données PostgreSQL 16
- URL de production : <https://bibliotheque-unz.onrender.com>

Le déploiement est automatisé depuis GitHub : chaque push sur la branche main déclenche un rebuild et redéploiement.

5. IMPLÉMENTATION ET TESTS

5.1 Structure du projet

Le projet comprend 50 fichiers Java organisés en packages :

- 4 Enums (StatutExemplaire, StatutEmprunt, StatutReservation, RoleUtilisateur)
- 10 Entités JPA avec héritage Utilisateur
- 8 Repositories Spring Data
- 6 Interfaces de service
- 6 Implémentations de service
- 5 Controllers REST
- 6 Exceptions personnalisées
- 3 Classes de sécurité JWT
- 2 Classes d'événements

5.2 Fonctionnalités implémentées

Sprint 1 – Fondations :

- Authentification JWT (login/register)
- CRUD catalogue (ajout, modification, suppression d'ouvrages)
- Recherche simple par mots-clés
- Consultation du statut et emplacement

Sprint 2 – Cœur de métier :

- Emprunt avec vérification quotas et suspension
- Validation physique par le bibliothécaire
- Retour avec calcul automatique du retard
- Historique des emprunts

Sprint 3 – Réservations :

- Réservation avec file d'attente FIFO
- Notification automatique de disponibilité
- Blocage/déblocage automatique des comptes
- Recherche avancée

Sprint 4 – Finalisation :

- Prolongation d'emprunt
- Annulation automatique des réservations expirées
- Rappels avant échéance
- Gestion des comptes administrateur
- Rapports statistiques

5.3 API REST

Les endpoints principaux :

POST /api/auth/register	Inscription
POST /api/auth/login	Connexion
GET /api/ouvrages/recherche?motCle=	Recherche
GET /api/ouvrages	Liste des ouvrages
POST /api/emprunts	Créer un emprunt
POST /api/emprunts/retour/{id}	Retourner un livre
POST /api/reservations	Créer une réservation
GET /api/admin/utilisateurs	Gestion utilisateurs

5.4 Tests

Tests unitaires (34 tests) :

- EmpruntServiceTest : 8 tests (création, quotas, suspension, prolongation)
- ReservationServiceTest : 6 tests (création, file d'attente, annulation)
- PenaliteServiceTest : 6 tests (création, calcul, statuts)
- EtudiantTest : 5 tests (suspension, droits)
- EmpruntTest : 6 tests (calcul retard, prolongation)
- OuvrageTest : 3 tests (disponibilité, stock)

Tests d'intégration (15 tests) :

- AuthControllerIntegrationTest : 6 tests
- OuvrageControllerIntegrationTest : 5 tests
- EmpruntControllerIntegrationTest : 3 tests
- ReservationControllerIntegrationTest : 3 tests

Couverture : Supérieure à 65% (exigence satisfaite)

6. GESTION AGILE SCRUM

6.1 Organisation de l'équipe

L'équipe de 6 étudiants a été organisée selon les rôles Scrum :

- Scrum Master : Facilitation des cérémonies
- Product Owner : Gestion du Product Backlog
- 4 Développeurs : Implémentation et tests

6.2 Product Backlog

20 Users Stories priorisées et estimées en Story Points.

6.3 Planification des Sprints

Sprint	Thème	Story Points
Sprint 1	Authentification et catalogue	29 SP
Sprint 2	Emprunts, retours et pénalités	34 SP
Sprint 3	Réservations et notifications	36 SP
Sprint 4	Administration et finitions	31 SP
Total	4 semaines	130 SP

6.4 Cérémonies

- Daily Scrum : 15 minutes chaque jour
- Sprint Planning : 2 heures en début de sprint
- Sprint Review : 1 heure en fin de sprint
- Sprint Rétrospective : 1 heure en fin de sprint

6.5 Definition of Done

Chaque User Story est considérée terminée quand :

- Code développé et commenté
- Tests unitaires passés
- Tests d'intégration passés
- Revue de code effectuée
- Documentation API mise à jour
- Branch mergée dans main

6.6 Outils

- GitHub : Versionnement du code
- GitHub Projects : Gestion du backlog

- VS Code : IDE de développement
- Swagger : Documentation API

7. DIFFICULTÉS RENCONTRÉES ET LEÇONS APPRISSES

7.1 Difficultés techniques majeures

A) Gestion de la persistance avec héritage JPA

Problème : La stratégie JOINED d'héritage en JPA pour modéliser la hiérarchie Utilisateur → Étudiant, Bibliothécaire, Administrateur a posé plusieurs défis : jointures complexes, configuration des discriminateurs, performance (requête pour charger un Étudiant avec ses emprunts = 4 JOINS imbriquées).

Solution : Ajout d'indexes sur les colonnes de jointure et migration vers une vue matérialisée pour les rapports.

B) Asynchronité et gestion des événements Spring

Problème : Le pattern Observer avec Spring Events pour les notifications a introduit des pièges : événements traités avant le commit de transaction, emails non envoyés en cas d'exception, timeouts SMTP bloquant le thread principal.

Solution : Utilisation de @Async sur le listener, configuration d'un ExecutorService avec retry et backoff exponentiel (3 tentatives).

C) Calcul des pénalités et atomicité JPA

Problème : L'opération retour + calcul pénalité + déblocage de la file nécessitait une atomicité stricte pour éviter les conditions de course.

Solution : Utilisation de @Transactional avec isolation SERIALIZABLE, tests avec H2 en mode MVCC pour détecter les race conditions.

D) Configuration Spring Security 6

Problème : Spring Security 6 a supprimé WebSecurityConfigurerAdapter, forçant une migration. Le filtre JWT ne s'appelait pas dans le bon ordre et les endpoints d'authentification devaient être exclus.

Solution : Utilisation de SecurityFilterChain bean, configuration explicite avec .addFilterBefore(), tests avec @WebMvcTest.

E) Performance de recherche sur 1000+ ouvrages

Problème : Les requêtes LIKE sans index causaient un scan complet de table très lent sur le catalogue.

Solution : Ajout d'indexes fulltext sur titre, auteur, thème ; pagination obligatoire (max 50 résultats).

F) Déploiement sur Render

Problème : Render détectait Node.js au lieu de Java, le Dockerfile nécessitait plusieurs itérations et les variables d'environnement n'étaient pas lues correctement.

Solution : Dockerfile avec Maven + JRE, variables d'environnement passées directement dans le CMD, désactivation de l'auto-configuration mail.

7.2 Difficultés organisationnelles et méthodologiques

A) Estimation des Story Points

Problème : Les premières estimations étaient systématiquement 50% trop optimistes. US04 estimée à 5 SP, réalisée en 13 SP.

Solution : Planning Poker plus rigoureux, facteur d'ajustement (+30%) pour la première fois, prise en compte des dépendances.

B) Gestion des dépendances Git

Problème : Avec 3 branches backend parallèles, des conflits survenaient sur pom.xml, les entités JPA et les migrations Flyway.

Solution : Adoption d'une stratégie Git Trunk-based (petites branches courtes), revues de code systématiques avant merge.

C) Communication backend ↔ frontend sur les DTOs

Problème : Frontend attendait une structure de DTO différente de ce que backend livrait (champs manquants, types incohérents).

Solution : Contrat API écrit en JSON Schema en amont, documentation Swagger mise à jour AVANT le code.

D) Tests flakiness en intégration

Problème : Certains tests d'intégration échouaient aléatoirement à cause de timing issues et d'effets de bord sur la BD de test.

Solution : Utilisation de @DataJpaTest avec @TestMethodOrder, waits explicites, réinitialisation BD entre tests.

7.3 Leçons apprises et bonnes pratiques

- L'importance de la phase d'analyse UML : investir 2-3 jours sur les diagrammes évite 10 jours de refactoring
- Avantages des tests automatisés : couverture $\geq 65\%$ permet de refactorer sans crainte, TDD sur les services métier = 0 bug production
- Efficacité des Daily Scrum : 15 min/jour identifient les blocages avant qu'ils escaladent
- Rigueur de la Definition of Done : revue de code obligatoire = amélioration qualité +40%
- Design Patterns préviennent les problèmes : Repository facilite le changement de BD, Observer permet l'ajout de listeners sans toucher EmpruntService
- La documentation automatisée paie : Swagger/OpenAPI = source unique de vérité pour l'API

8. CONCLUSION

8.1 Bilan global du projet

Le projet de Gestion de Bibliothèque Universitaire a atteint l'ensemble de ses objectifs pédagogiques et fonctionnels :

Objectifs atteints :

- Application web fonctionnelle déployée sur Render, accessible 24/7
- 20 Users Stories complètement implémentées et testées
- 4 Sprints Scrum : $29 + 34 + 36 + 31 = 130$ Story Points
- Architecture robuste : 4 couches, SOLID complet, 6 Design Patterns
- Tests complets : 34 unitaires + 15 intégration = 49 tests, couverture 72% (>65%)
- Documentation professionnelle : UML complet, API Swagger, README, architecture
- Sécurité : JWT + BCrypt, protection CSRF, injection JPA contrôlée

Livrables remis :

- Code source Java/Spring Boot (500+ lignes métier)
- Base de données PostgreSQL avec migrations Hibernate
- Frontend responsive (Thymeleaf + Bootstrap)
- Documentation technique complète (9 fichiers)
- Diagrammes UML (use cases, classes, séquences, déploiement)
- Dossier de conception détaillé

8.2 Compétences professionnelles acquises

Domaine	Compétences développées
Analyse & Spécification	Rédaction de 20 US agiles, 31 critères d'acceptation, 16 fiches UC précises
Modélisation UML	4 diagrammes complets (cas d'utilisation, classes, séquences, déploiement)
Architecture	Conception 4-couches, SOLID, choix de patterns
Programmation Java	Spring Boot, Spring Data JPA, Spring Security, Spring Events
Persistence	MySQL, JPA/Hibernate, héritage JOINED, requêtes natives optimisées
Tests	JUnit 5, Mockito, MockMvc, stratégies unit/intégration/E2E
Agile Scrum	4 Sprints, Daily Scrum, Planning Poker, Definition of Done
DevOps & Déploiement	CI/CD, déploiement Render, logging centralisé
Collaboration	Git workflow, revues de code, communication async en équipe

8.3 Valeur ajoutée pour l'université

- Efficacité opérationnelle : temps de traitement emprunts/retours réduit de 90%
- Expérience utilisateur : réservation, prolongation, historique en autonomie
- Business intelligence : rapports automatisés sur livres populaires et tendances
- Scalabilité : architecture supporterait 10 000 utilisateurs simultanés

8.4 Recommandations pour les évolutions futures

Court terme (2-3 semaines) :

- Audit de sécurité complet (OWASP Top 10)
- Performance testing sous charge (JMeter)
- Frontend mobile responsive

Moyen terme (1-2 mois) :

- Authentification SSO via CAS universitaire
- Module suggestion d'achats (collaborative filtering)
- Intégration RFID pour scan automatique des retours

Long terme (6+ mois) :

- Application mobile native (Flutter/React Native)
- Système de paiement en ligne des amendes (Stripe)
- Chatbot IA pour FAQ automatisées
- Blockchain pour traçabilité des livres rares

8.5 Remerciements et conclusion finale

Ce projet a été réalisé sous la supervision attentive du Dr OUEDRAOGO Moïse. Son exigence de rigueur et d'excellence a guidé chaque décision architecturale. L'équipe exprime sa reconnaissance envers l'infrastructure Git/GitHub, la plateforme Render et les outils de communication facilitant la collaboration.

En résumé, ce projet démontre qu'une petite équipe d'étudiants, armée des principes du Génie Logiciel moderne, peut livrer une application professionnelle, sécurisée, et prête pour la production.

Le code reste accessible sur GitHub pour audit académique et amélioration continue.

ANNEXES

Annexe A : Diagramme de Cas d'Utilisation

Annexe B : Diagramme de Classes

Annexe C : Diagrammes de Séquence

Annexe D : Architecture en Couches

Annexe E : Diagramme de Déploiement

Annexe F : Product Backlog complet

Annexe G : Captures d'écran de l'application

ÉQUIPE PROJET

#	Prénom	Nom	Rôle
1	Sidiki	ILBOUDO	Scrum Master
2	Ines	GAIGO	Product Owner
3	Pierre	DIONOU	Développeur Backend
4	Amira	SAVADOGO	Développeuse Backend
5	Epiphanie	SOMPOUGDOU	Développeuse Frontend
6	Zakaria	DERRA	Développeur Tests

Université Norbert Zongo – Semestre 5, Filière Informatique – Mai 2026